Sim2Value: Transferring Role-Aligned Object-Centric Representations for Real-World Value Learning

Geunha Kim, Haotian Zhang, and Jun Moon*

***Abstract*— Visual and physical domain gaps compound over a rollout, driving simulation-trained policies into states outside their training distribution. Rather than transferring a control policy, we use simulation to pretrain an object-centric visual encoder for real-world value learning. The recurrent encoder represents the robot, instruction-specified object, and goal as role-aligned slots. Tracks propagated from initial-frame bounding boxes supervise these slots throughout the trajectory, preserving their assigned roles without dense mask annotations. As few as five successful real trajectories per task suffice to adapt the encoder. The real trajectories alone are not enough: without simulation pretraining, value-order correlation falls from 77.3 to 53.1. Against global image representations and a smoothing control, role-aligned slots are the only representation that combines low spurious value decreases (13.5% of frames) with a clear separation of object drops from ordinary variation. Offline advantage labels derived from these values raise real-robot success from 23.3% to 45.0%, so routing simulation pretraining through perception improves a real-world policy without transferring control.**

I. Introduction

Prior work transfers simulation-trained robot policies to real environments through visual and dynamics randomization [1], [2] and image translation [3]. Transfer nonetheless remains difficult. An observation-to-action mapping is sensitive to discrepancies in dynamics, contact, and actuation. Closed-loop execution compounds this: small deviations accumulate over a rollout, driving the system toward states not covered during training, where the mapping degrades.

Rather than transferring a policy, we pretrain a visual state encoder in simulation and adapt it to real observations (Fig. 1), then train the value model and policy on real trajectories. The resulting values provide offline labels for Vision-Language-Action (VLA) model fine-tuning [4], [5]. This routes simulation pretraining through perception rather than control, where the domain gap is primarily visual.

Pretraining in simulation does not by itself resolve the visual domain gap. Global image representations encode the entire scene jointly, allowing domain-specific differences in appearance, illumination, and background to alter the representation even when the underlying object configuration is unchanged. We therefore ask whether explicit task-role structure improves value learning compared with global image embeddings [6]–[8]. Object-centric models [9]–[11] provide entity-level representations, but tracking an entity over time does not tell the value model which entity the instruction refers to. Our object-centric encoder supervises robot, instructed-object, and goal assignments explicitly, making entity states and inter-role relationships directly accessible to the value model.

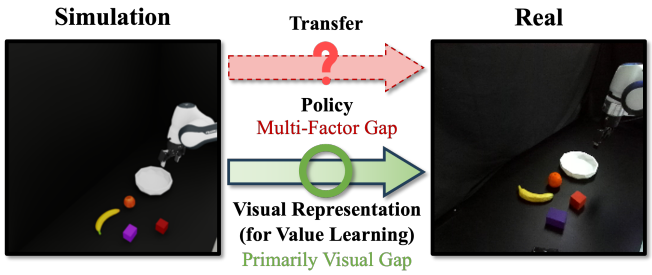


Fig. 1. Conceptual overview. In contrast to direct policy transfer, our approach transfers a simulation-pretrained visual representation and adapts it for real-world value learning.

To maintain these role assignments over time, we introduce tracker-guided trajectory-level supervision. Role boxes are annotated only in the initial frame of each trajectory and propagated by a pretrained point tracker [12]. These tracks yield spatial pseudo-labels that supervise the recurrent slots throughout the trajectory, without dense mask annotations. We pretrain a shared role encoder in simulation and adapt it on a small number of real trajectories. An embedding from a frozen language encoder [13] conditions the object query, letting the encoder ground the instructed object.

The adapted slots, together with the same instruction embedding, form the state for the value model. It is trained with a RECAP-style distributional objective [5] and used offline: its predictions become binary advantage conditions on recorded action chunks, which we use to fine-tune a flow-based VLA policy [4]. Across four target-object variants of a multi-object pick-and-place task, role-aligned slots combine temporally consistent values with sensitivity to object drops, which no evaluated image representation achieves together. These values raise real-robot success from 23.3% to 45.0%. Simulation thus contributes through perception, without a control policy ever crossing the domain gap.

Our contributions are:

- We formulate sim-to-real representation transfer for real-world value learning by pretraining role-structured representations in simulation and adapting them with limited real data.
- We introduce instruction-conditioned, tracker-guided recurrent role slots that use propagated spatial pseudo-labels to preserve temporally consistent task-role identities and ground the instruction-specified object.
- We evaluate temporal value consistency and object-drop sensitivity against image, dimensionality, ordering, and

*Corresponding author

pseudo-label controls, showing that role-aligned slots combine both where the controls do not, and that the resulting offline conditions raise real-robot success.

II. Related Work

A. Sim-to-Real Robot Learning

Randomization [1], [2], image translation [3], and domain-invariant representations [14] address visual and physical discrepancies between simulation and reality. Transferred policies remain vulnerable to compounding errors [15] during closed-loop execution. A value model scores recorded states rather than executing them, so it faces only the visual gap. We therefore use simulation to pretrain visual representations, while learning the value model and policy from real data. Existing pretrained manipulation representations [7], [8] encode scenes globally; our focus is whether explicit, instruction-dependent role structure provides a more reliable state representation for real-world value learning.

B. Object-Centric Representation Learning

Slot Attention [9] decomposes scenes into exchangeable slots, and SAVi [10] propagates them through video by reconstructing pixels. DINOSAUR [11] instead reconstructs self-supervised features, which plausibly transfer better by reducing dependence on low-level appearance. Exchangeability is the deeper obstacle. Temporal objectives [16], [17] and conditioning cues [18] keep a slot on the same entity, but they do not fix which slot should represent that entity, and the instruction decides which one the value model needs. Object-centric states support control [19]–[21], and open-vocabulary perception grounds instructions per frame [22]–[24]. We instead require one slot to retain its referent throughout. Rather than annotating masks densely [18] or running a detector [23], [24], we propagate initial-frame boxes offline and distill them into the encoder, giving the value model a stable role-indexed state from observations alone. This reduces spurious value changes from identity switches.

C. Value Learning and Advantage-Conditioned VLA Policies

VLA models learn instruction-conditioned policies from demonstrations [4], [25], [26]. Value learning supports policy improvement from offline data [27], while return conditioning [28] and advantage-weighted regression [29] guide policies toward higher-return behavior. RECAP [5] combines distributional value learning with advantage-conditioned VLA training. These methods take the visual state as given. That state matters: in offline learning, advantages are determined by value differences across recorded states. A representation that shifts with appearance rather than with task progress therefore produces misleading policy labels, whereas excessive smoothing suppresses responses to failures. We therefore ask how to structure that state: our simulation-pretrained role slots encode each task-relevant entity separately under a fixed role, so the state changes when those entities move rather than when scene appearance does. We show that this yields temporally consistent yet object-drop-sensitive values that supply offline advantage labels.

III. Sim-to-Real Object-Centric Representation Learning with Tracker Guidance

A. Role-Aligned State Representation

Given a visual trajectory $\tau = \{x_t\}_{t=1}^{T}$, where x_t denotes the RGB frame at time t, and instruction l, the recurrent encoder updates

$$S_t = f_\theta(x_t, S_{t-1}, l) = \left(z_t^{\mathrm{bg}}, z_t^r, z_t^o, z_t^g\right),$$

where the superscripts bg, r, o, and g denote the background, robot, object, and goal, respectively. A frozen language encoder [13] provides an instruction feature, which a trainable projection maps to the slot dimension. At each Slot Attention iteration, this vector is added only to the object slot. The value model uses the foreground state $z_t = [z_t^r; z_t^o; z_t^g]$.

B. Tracker-Guided Role Grounding

Only the initial frame of each trajectory is annotated with role-level bounding boxes. As shown in Fig. 2(a), our primary method samples role-specific point queries from these boxes and propagates them using CoTracker [12]. The valid tracks are converted into Gaussian heatmaps H_t^k for $k \in \{r, o, g\}$, as illustrated in Fig. 2(b). These heatmaps provide approximate spatial pseudo-labels without requiring complete object boundaries.

To examine whether dense support provides better supervision, we also evaluate SAM 2 [30], initialized from the same boxes, as an alternative offline pseudo-label source in Sec. V-C. Both serve only as offline supervision; at deployment, the encoder requires only the observation and instruction.

Let M_t^k be the soft mask predicted by role slot k. We normalize the heatmap H_t^k and mask over the spatial grid Ω:

$$Q_t^k(p) = \frac{H_t^k(p)}{\sum_{p' \in \Omega} H_t^k(p') + \varepsilon}, \qquad P_t^k(p) = \frac{M_t^k(p)}{\sum_{p' \in \Omega} M_t^k(p') + \varepsilon},$$

where ε is a small constant for numerical stability.

The distribution-alignment loss is

$$\mathscr{L}_{\mathrm{dist}} = -\mathbb{E}_{(t,k)\in\mathscr{V}} \sum_{p \in \Omega} Q_t^k(p) \log\left(P_t^k(p) + \varepsilon\right),$$

where $\mathscr{V}$ contains the time–role pairs for which the tracker produces a valid, non-empty heatmap. We additionally use role competition and mask regularization:

$$\mathscr{L}_{\mathrm{ground}} = \mathscr{L}_{\mathrm{dist}} + \lambda_{\mathrm{comp}} \mathscr{L}_{\mathrm{comp}} + \lambda_{\mathrm{mask}} \mathscr{L}_{\mathrm{mask}}.$$

$\mathscr{L}_{\mathrm{comp}}$ compares how much mask mass each slot places inside a tracked region, and penalizes any competing slot—including the background—that comes within a margin of the correct role slot. $\mathscr{L}_{\mathrm{mask}}$ suppresses foreground mass outside the tracked regions and encourages compact, smooth masks. Loss weights are given in Sec. V.

C. Recurrent Role Slot Propagation

We use truncated backpropagation through time over consecutive, non-overlapping chunks of W frames, preserving recurrent slot states across chunks. Within each chunk, a feature-conditioned transition initializes Slot Attention:

$$\widetilde{S}_{c,i} = g_\psi(S_{c,i-1}, F_{c,i}), \qquad S_{c,i} = \mathrm{SlotAttention}\left(F_{c,i}, \widetilde{S}_{c,i}; l\right),$$

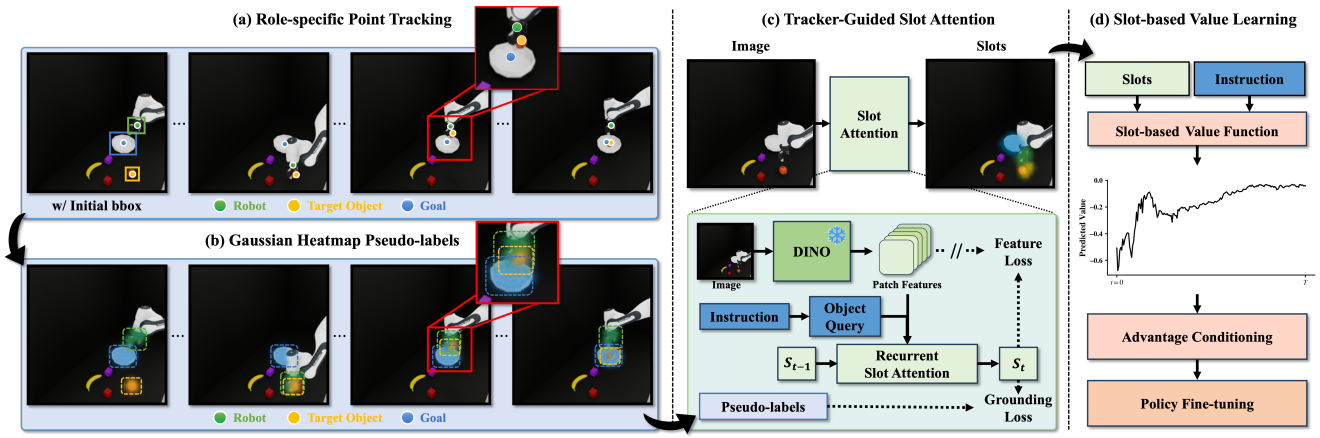


Fig. 2. Overview of instruction-conditioned, tracker-guided role slot learning. **(a)** Role-specific points initialized in the first frame are propagated through simulation trajectories using CoTracker. **(b)** The propagated points are converted into Gaussian heatmaps that provide spatial pseudo-labels for the robot, object, and goal. **(c)** The instruction conditions the object query, while spatial pseudo-labels and feature reconstruction supervise recurrent role slot learning. **(d)** The resulting role slots and instruction are used for value learning and advantage-conditioned policy fine-tuning.

where c indexes chunks, i indexes frames within a chunk, and $F_{c,i}$ denotes visual features. The transition g_ψ attends from each previous slot to current visual tokens, followed by a shared per-slot GRU and residual MLP. Slots are initialized from learned embeddings at each trajectory start, and l conditions only the object query.

At each frame, masks decoded from $S_{c,i}$ are supervised by $\mathscr{L}_{\text{ground}}$ together with the representation objective introduced next. At each chunk boundary, the final slot state is propagated while its gradient is truncated:

$$S_{c,0} = \text{stopgrad}(S_{c-1,W}).$$

Slots are reset only at trajectory boundaries.

D. Sim-to-Real Representation Learning

We learn the visual content of each role slot by reconstructing frozen DINO patch features [6], following DINOSAUR [11]. Compared with RGB pixels, these features are less coupled to low-level appearance variation, making them suitable targets for sim-to-real representation learning. We compare this formulation with an RGB-reconstructing SAVi-style baseline in Sec. V.

Let $F_t(p) \in \mathbb{R}^D$ denote the frozen DINO feature at patch location p, and let $\widehat{F}_t(p)$ denote its slot-decoded reconstruction. The decoder predicts a feature vector and mask logit for each slot and patch. A softmax over all slots, including background, produces masks whose weighted feature sum gives $\widehat{F}_t(p)$. We normalize both features as

$$f_t(p) = \frac{F_t(p)}{\|F_t(p)\|_2 + \varepsilon}, \qquad \widehat{f}_t(p) = \frac{\widehat{F}_t(p)}{\|\widehat{F}_t(p)\|_2 + \varepsilon},$$

and minimize the feature reconstruction loss

$$\mathscr{L}_{\text{feat}} = \mathbb{E}_{t,\, p \in \Omega_F} \left\| \widehat{f}_t(p) - f_t(p) \right\|_2^2,$$

where Ω_F is the DINO patch grid.

We jointly train the instruction-conditioned recurrent slots using both tracker-guided grounding and feature reconstruction losses (Fig. 2(c)):

$$\mathscr{L}_{\text{slot}} = \lambda_{\text{ground}} \mathscr{L}_{\text{ground}} + \lambda_{\text{feat}} \mathscr{L}_{\text{feat}}.$$

We then adapt the simulation-pretrained model on K real trajectories per task using the same objective; we use $K = 5$ (Sec. V). During adaptation, the PaliGemma [13] language encoder remains frozen, while the instruction-to-query projection and role slot encoder are updated jointly.

IV. Object-Centric Value Learning and Advantage-Conditioned VLA Fine-Tuning

A. Language-Conditioned Object-Centric Value Learning

We freeze the adapted encoder and use its foreground state z_t for value learning. The instruction l is encoded by the frozen language encoder and provided to the value network together with z_t. Thus, the same instruction conditions both object selection and task-value prediction.

We use a RECAP-style distributional return objective [5] with B uniformly spaced bins $\{q_j\}_{j=1}^B$ over $[-1, 0]$. Let $T_l^{\max}$ be the maximum number of transitions among training trajectories associated with instruction l. The return target is

$$\hat{G}_t = \begin{cases} -\dfrac{T_\tau - t}{T_l^{\max}}, & \tau \text{ is successful,} \\ -1, & \tau \text{ is unsuccessful,} \end{cases}$$

where T_τ indexes the terminal state. Successful trajectories receive a normalized remaining-step target, whereas failed trajectories receive a constant outcome target of -1, not a cumulative step-cost return. Failure targets supervise value learning only; policy labels are constructed from successful demonstrations. This supervision does not explicitly mark the onset of a failure.

The network predicts $p_\phi(q_j \mid z_t, l)$ and is trained with cross-entropy against the bin nearest $\hat{G}_t$. Its scalar value is

$$V_\phi(z_t, l) = \sum_{j=1}^{B} q_j p_\phi(q_j \mid z_t, l).$$

As illustrated in Fig. 2(d), the resulting values are used to construct offline advantage labels for VLA policy fine-tuning.

TABLE I

SIMULATION ROLE GROUNDING ON 40 HELD-OUT TEST TRAJECTORIES FOR A SINGLE TRAINING SEED. (METRICS SCALED BY 100; MEAN ± STANDARD DEVIATION ACROSS TRAJECTORIES). BOLDFACE MARKS THE BEST MEAN ACROSS ALL ROWS FOR EACH METRIC.

Backbone	BBox Cond.	Tracker Sup.	Instr. Query	AP@6px ↑			NCE ↓		
				Robot	Object	Goal	Robot	Object	Goal
SAVi [10]	×	×	×	1.3 ± 0.1	1.1 ± 0.9	3.3 ± 0.2	19.0 ± 2.7	23.8 ± 4.2	12.5 ± 2.1
	✓	×	×	2.1 ± 0.2	1.0 ± 0.2	3.3 ± 0.2	17.9 ± 2.7	39.0 ± 9.3	18.4 ± 0.6
	×	✓	×	87.8 ± 9.8	55.0 ± 25.9	94.9 ± 4.5	1.3 ± 0.7	3.9 ± 3.8	0.7 ± 0.3
	×	✓	✓	**94.1 ± 3.4**	**83.3 ± 19.2**	**96.0 ± 2.7**	**0.9 ± 0.4**	2.2 ± 4.2	**0.5 ± 0.2**
DINOSAUR [11]	×	×	×	13.4 ± 2.3	4.4 ± 1.0	2.0 ± 0.2	17.3 ± 2.4	22.5 ± 4.0	18.5 ± 0.9
	✓	×	×	2.1 ± 0.2	1.0 ± 0.2	3.1 ± 0.1	17.5 ± 2.5	22.5 ± 4.0	17.8 ± 0.7
	×	✓	×	86.7 ± 4.9	66.6 ± 17.6	86.8 ± 5.1	1.4 ± 0.5	2.5 ± 3.3	0.9 ± 0.3
	×	✓	✓	90.8 ± 2.7	70.7 ± 14.3	90.4 ± 4.0	1.1 ± 0.4	**2.0 ± 1.7**	0.8 ± 0.2

B. Advantage-Conditioned Policy Fine-Tuning

We compute offline n-step return residuals from successful demonstrations. Let T_τ index the terminal state and $h_t = \min(n, T_\tau - t)$. Using the same instruction-specific normalization as value learning, each transition before the terminal state incurs $r_t = -1/T_l^{\max}$, and the terminal-state value is set to zero. The n-step residual is

$$A_t^{(n)} = -\frac{h_t}{T_l^{\max}} + \mathbb{I}[t + h_t < T_\tau] V_\phi(z_{t+h_t}, l) - V_\phi(z_t, l).$$

Following RECAP [5], this undiscounted residual vanishes when the predicted values match the successful-trajectory targets, so it measures how much a transition outpaces the linear progress those targets assume. Training the value model on failed as well as successful trajectories keeps these deviations tied to task-relevant state changes.

We standardize these residuals using instruction-specific statistics and convert their signs into binary conditions:

$$\widetilde{A}_t = \frac{A_t^{(n)} - \mu_l}{\sigma_l + \varepsilon}, \qquad c_t = \begin{cases} +1, & \widetilde{A}_t > 0, \\ -1, & \widetilde{A}_t \le 0. \end{cases}$$

Thus, $c_t = +1$ and $c_t = -1$ denote action chunks with advantages above and at or below the instruction-specific mean, respectively. The condition is embedded into the flow-timestep representation of the VLA action expert. We optimize equally weighted conditioned and unconditioned flow-matching objectives and use $c_t = +1$ at inference. The value model and role slots are used only to annotate recorded action chunks offline.

V. EXPERIMENTS

Our experiments evaluate role grounding in simulation and on real observations, the contribution of simulation pretraining, the temporal reliability of the resulting value, and its effect on real-robot policy performance.

A. Common Experimental Setup

Tasks and datasets. We study four target-object variants of a multi-object pick-and-place task: orange, banana, red cube, and purple cube. Simulation uses 160 training and 40 held-out trajectories. The real training set contains five successful trajectories per task (20 trajectories; 2,858 frames), used for role-encoder adaptation, value learning, and policy fine-tuning. Value learning also uses 16 failure trajectories (3,165 frames), equally divided among object drops, grasp misses, wrong-object interactions, and stalled motions.

A separate set of 20 successful trajectories with frame-wise role masks is split into four validation and 16 test trajectories, and eight additional object-drop trajectories are reserved for value evaluation. All evaluation trajectories are excluded from role-encoder and value-model training. For real adaptation, we annotate three role boxes in only the initial frame of each training trajectory (60 boxes total). These annotations are unavailable during evaluation and deployment.

Implementation details. We use 224×224 images, recurrent chunks of $W = 4$ frames, and $B = 201$ return bins. Observations and actions are recorded at 10 Hz, so the $n = 16$ value horizon spans 1.6 seconds and matches the policy's 16-step action horizon. Slot-learning weights are $\lambda_{\text{ground}} = 1$, $\lambda_{\text{feat}} = 0.1$, $\lambda_{\text{comp}} = 2$, and $\lambda_{\text{mask}} = 1$, with a competition margin of 0.1. For tracker supervision we sample 25 points per role, keep up to 12 motion-consistent tracks with at least three valid points, and rasterize them with four-pixel Gaussian kernels. Role encoders and value models are trained with three seeds, except for the single-seed simulation comparison in Table I. Policy evaluation uses three inference seeds.

Simulation data. Simulation trajectories are collected in NVIDIA Isaac Sim using a rule-based policy. Object poses, camera pose, and lighting are randomized during collection.

Hardware. Experiments use four NVIDIA RTX A5000 GPUs, a Franka Research 3 with a Robotiq 2F-85 gripper, and one side-view ZED 2i camera.

B. Simulation Role Grounding

Evaluation protocol. Table I compares RGB-reconstructing SAVi-style and feature-reconstructing DINOSAUR-style encoders. *BBox Cond.* denotes first-frame bounding box conditioning of slot initialization; *Tracker Sup.* denotes supervision with CoTracker-derived pseudo-labels; and *Instr. Query* denotes instruction conditioning of the object query. Rows without these components use reconstruction alone.

Following the use of pixel-wise average precision (AP) in segmentation benchmarks [31], [32], we define AP@6px as a tolerance variant computed against ground-truth masks

TABLE II

REAL-WORLD ROLE GROUNDING WITH COTRACKER OR SAM 2 OFFLINE SUPERVISION (16 HELD-OUT TEST TRAJECTORIES; AP AND NCE SCALED BY 100, MEAN ± STANDARD DEVIATION ACROSS THREE TRAINING SEEDS). BOLD/UNDERLINE MARK THE BEST OVERALL/ZERO-SHOT RESULTS.

Backbone	Instr. Query	Training	CoTracker						SAM 2					
			AP@6px ↑			NCE ↓			AP@6px ↑			NCE ↓		
			Robot	Object	Goal	Robot	Object	Goal	Robot	Object	Goal	Robot	Object	Goal
SAVi	×	Sim only	10.4 ± 2.1	10.3 ± 1.0	44.4 ± 2.6	11.8 ± 0.7	10.2 ± 1.1	5.3 ± 1.3	11.9 ± 5.6	9.3 ± 1.3	58.5 ± 11.8	10.8 ± 1.2	9.5 ± 0.8	3.3 ± 1.6
		Real only	28.2 ± 18.5	21.4 ± 10.5	85.9 ± 9.7	7.3 ± 2.6	8.3 ± 2.4	1.8 ± 1.2	28.6 ± 15.8	19.9 ± 10.9	67.5 ± 5.4	6.5 ± 2.0	8.2 ± 2.4	1.8 ± 1.1
		Sim + Real	17.9 ± 1.5	15.0 ± 1.9	86.7 ± 5.5	8.2 ± 0.2	9.3 ± 1.0	1.3 ± 0.9	24.0 ± 3.1	17.0 ± 0.2	65.6 ± 2.5	7.3 ± 0.2	7.9 ± 0.2	1.5 ± 1.2
	✓	Sim only	18.7 ± 2.3	19.3 ± 11.2	57.7 ± 16.2	7.5 ± 2.0	7.9 ± 1.5	3.4 ± 0.4	11.3 ± 1.3	9.3 ± 1.8	55.0 ± 10.6	11.2 ± 0.5	10.0 ± 0.7	4.4 ± 1.3
		Real only	37.2 ± 15.3	17.5 ± 1.5	92.9 ± 4.6	5.4 ± 1.9	8.8 ± 0.7	0.9 ± 0.6	36.1 ± 17.0	17.5 ± 3.9	65.7 ± 8.6	5.3 ± 2.4	7.3 ± 1.3	1.1 ± 0.3
		Sim + Real	49.3 ± 3.5	27.0 ± 2.5	90.6 ± 0.4	**2.6 ± 0.3**	7.0 ± 0.1	1.1 ± 0.0	27.1 ± 2.0	17.8 ± 0.7	68.3 ± 4.1	6.9 ± 0.4	7.9 ± 0.7	**0.8 ± 0.2**
DINOSAUR	×	Sim only	21.0 ± 2.9	12.3 ± 1.1	47.9 ± 7.8	7.2 ± 0.7	8.5 ± 0.5	4.6 ± 0.6	21.5 ± 2.7	13.8 ± 6.2	42.6 ± 5.7	6.8 ± 1.1	8.7 ± 2.1	5.1 ± 0.8
		Real only	45.3 ± 3.9	26.7 ± 2.5	79.2 ± 5.4	4.9 ± 0.1	5.9 ± 0.8	2.7 ± 0.3	46.9 ± 5.2	27.9 ± 3.2	69.4 ± 6.2	4.6 ± 0.4	5.1 ± 0.7	3.8 ± 0.7
		Sim + Real	65.1 ± 5.0	24.8 ± 4.3	92.7 ± 1.5	3.0 ± 0.4	6.1 ± 0.7	1.2 ± 0.1	53.8 ± 6.5	35.0 ± 1.3	69.9 ± 5.2	3.7 ± 0.9	4.1 ± 0.5	3.2 ± 0.7
	✓	Sim only	32.2 ± 15.9	20.4 ± 0.9	52.8 ± 16.6	6.3 ± 2.5	7.4 ± 1.9	4.6 ± 1.3	23.4 ± 13.1	16.9 ± 6.8	34.7 ± 17.8	7.4 ± 2.9	6.3 ± 0.8	6.9 ± 2.6
		Real only	43.5 ± 5.2	24.2 ± 2.8	80.5 ± 4.5	5.0 ± 0.4	6.2 ± 0.3	2.7 ± 0.5	43.9 ± 6.2	31.5 ± 5.7	72.7 ± 2.6	4.7 ± 0.4	4.5 ± 0.5	3.4 ± 0.3
		Sim + Real	**69.3 ± 2.0**	**50.0 ± 3.1**	**96.4 ± 0.4**	2.7 ± 0.2	4.1 ± 0.3	**0.8 ± 0.1**	54.0 ± 10.4	46.6 ± 15.1	72.3 ± 9.6	3.8 ± 1.4	**2.9 ± 1.1**	3.1 ± 1.2

dilated with a six-pixel-radius elliptical kernel. The tolerance accommodates the patch-grid resolution at which slot masks are decoded, since we measure whether a slot lands on the correct role rather than its boundary precision. Predicted soft-mask values provide pixel confidence, so the metric is threshold-free and distinct from detection AP.

Normalized center error (NCE) measures the distance between the predicted and ground-truth mask centers, normalized by the image diagonal:

$$\mathrm{NCE}^k = \frac{\left\| c(\hat{M}^k) - c(G^k) \right\|_2}{\sqrt{H_{\mathrm{img}}^2 + W_{\mathrm{img}}^2}}, \qquad c(A) = \frac{\sum_p pA(p)}{\sum_p A(p) + \varepsilon},$$

where $\hat{M}^k$ and G^k denote the predicted and ground-truth masks for role k, and $c(A)$ is the mass center of a mask A. We exclude the initial frame and frames with empty role masks. AP pools pixels across valid frames within each trajectory using 256 confidence bins, whereas NCE averages frame-level errors. Table I reports trajectory-level mean and standard deviation, scaled by 100. Tracker pseudo-labels are never used as evaluation ground truth; masks come from simulator instance annotations.

Results. Reconstruction-only and initial-frame box baselines obtain low fixed-role grounding scores on subsequent frames, while trajectory-level supervision substantially improves these scores. Tracker pseudo-labels increase DINOSAUR robot/object/goal AP@6px from 13.4/4.4/2.0 to 86.7/66.6/86.8 and reduce NCE from 17.3/22.5/18.5 to 1.4/2.5/0.9, with similarly large gains for SAVi. Instruction conditioning increases object AP@6px from 55.0 to 83.3 for SAVi and from 66.6 to 70.7 for DINOSAUR. The object remains the most challenging role because of competing distractors and frequent gripper occlusion.

C. Real-World Role Grounding

Evaluation protocol. Real-world ground-truth masks are manually annotated, so we again report AP@6px and NCE. Sim-only models are trained for 100 simulation epochs, Real-only models for 75 real epochs, and Sim+Real models are initialized from the 100-epoch simulation checkpoint and adapted for 75 real epochs without simulation replay. Within each training regime, variants with and without instruction conditioning use identical budgets. For each seed, metrics are averaged over the 16 test trajectories, and we report the mean and standard deviation across three seeds.

Results. Table II compares matched models trained with CoTracker- and SAM 2-derived pseudo-labels. With CoTracker, instruction conditioning raises Sim+Real DINOSAUR object AP@6px from 24.8 to 50.0, whereas the gain is absent under Real-only training. For the instruction-conditioned Sim+Real model, CoTracker increases role-averaged AP@6px from 57.6 to 71.9 and reduces mean NCE from 3.3 to 2.5 relative to SAM 2, although SAM 2 obtains lower object NCE (2.9 versus 4.1). We therefore use CoTracker as the primary supervision source and retain SAM 2 for the value ablation in Table IV.

D. Simulation Pretraining and Real Adaptation

Evaluation protocol. We examine how simulation pretraining affects real adaptation using the instruction-conditioned DINOSAUR encoder. Simulation checkpoints up to 500 epochs are evaluated zero-shot, while the 25-, 50-, 100-, and 200-epoch initializations are adapted for up to 200 real epochs. All runs use five training trajectories per task and the four-trajectory validation split.

Effect of pretraining duration. Fig. 3(a–b) reveals a trade-off across roles: longer pretraining improves object grounding but can reduce robot grounding, with non-monotonic changes beyond 200 epochs. We therefore select the pretraining duration using adaptation performance and computational cost rather than zero-shot role averages.

Effect on real adaptation. Fig. 3(c–d) shows that simulation-pretrained models adapt faster than Real-only training. Table III reports the role-specific validation results after 75 adaptation epochs. Sim 100 achieves an average AP@6px of 76.6, within 0.5 points of Sim 200, while requiring half as many pretraining epochs. We therefore select Sim 100 as the earliest near-saturated checkpoint and use 75 real adaptation epochs in subsequent experiments.

E. Value Reliability and Object-Drop Sensitivity

Evaluation protocol. We compare value functions using different visual representations. Image baselines use pooled DINO [6], R3M [7], or VIP [8] features. PCA variants reduce inputs to 192 dimensions, matching the dimensionality of

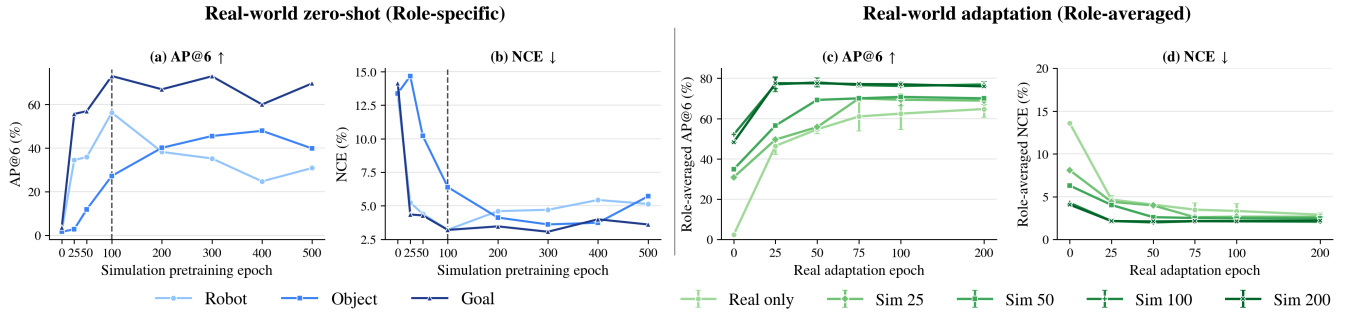


Fig. 3. Simulation-pretraining and real-adaptation analysis on the four-trajectory validation split. **(a–b)** Role-specific zero-shot AP@6px and NCE across simulation-pretraining epochs; the dashed line marks the epoch-100 checkpoint selected for real adaptation. **(c–d)** Role-averaged AP@6px and NCE during real adaptation from different simulation initializations. Error bars denote standard deviation across three seeds.

TABLE III

VALIDATION AP@6PX BY ROLE AFTER 75 REAL EPOCHS (%, MEAN ± STANDARD DEVIATION, THREE SEEDS). SIM N: N SIMULATION PRETRAINING EPOCHS. BOLDFACE MARKS THE BEST SCORE.

Init.	Robot	Object	Goal	Avg.
Real only	60.0 ± 15.6	35.8 ± 3.0	87.4 ± 9.2	61.1 ± 7.3
Sim 50	75.1 ± 3.5	38.0 ± 2.5	**97.0 ± 0.1**	70.1 ± 1.2
Sim 100	**77.3 ± 0.6**	55.8 ± 2.9	96.7 ± 0.3	76.6 ± 1.0
Sim 200	76.9 ± 0.6	**57.5 ± 2.3**	96.9 ± 0.3	**77.1 ± 0.8**

the role slots but not their recurrent history. For Role-shuffled, we permute the three foreground slots at each frame after recurrent feature extraction, testing the value head's sensitivity to slot ordering while preserving the encoder's temporal information. All value models share the same training setup; only their representations differ. For the simulation-initialization comparison, the Real-only encoder is trained for 200 epochs—longer than the 75-epoch Real-only models in Table II and the 175 encoder-training epochs of the Sim+Real encoder (100 sim and 75 real). DINO values with causal exponential moving average (EMA) ($\alpha = 0.5$) serve as a smoothing control.

Three metrics assess value reliability on 16 held-out successful trajectories. Following GVL [33], value-order correlation (VOC) measures the Spearman correlation between value and trajectory progress. Bump Ratio is the fraction of consecutive frames whose value decreases by more than 1% of the return range, and Bump Magnitude measures mean normalized downward variation.

Two metrics assess object-drop sensitivity on eight held-out failure trajectories. Drops are the one failure mode with an unambiguous onset frame, which the event windows require. Event Response is the mean value over the five frames preceding the annotated drop minus that over the five frames beginning at the drop. We contextualize it with the event signal-to-noise ratio (SNR),

$$\mathrm{SNR}_{\mathrm{event}} = \frac{R_{\mathrm{event}} - \mu(\mathscr{R}_{\mathrm{normal}})}{\sigma(\mathscr{R}_{\mathrm{normal}}) + \varepsilon},$$

where $\mathscr{R}_{\mathrm{normal}}$ collects responses computed with the same five-frame windows at one-frame stride, excluding centers within ten frames of the drop, and $\mu(\cdot)$ and $\sigma(\cdot)$ denote mean and standard deviation of $\mathscr{R}_{\mathrm{normal}}$. All metrics are reported as mean and standard deviation across three value-model seeds.

Representation comparison. Table IV shows different trade-offs. VIP-PCA matches role slots on the successful-trajectory metrics (VOC 76.0 versus 77.3, Bump Ratio 14.4 versus 13.5) but responds weakly at object drops (Event SNR 0.43 versus 2.91), whereas DINO-PCA responds more strongly than VIP-PCA yet decreases spuriously on 31.9% of frames. No image baseline achieves both, whereas role slots combine a VOC of 77.3 and Bump Ratio of 13.5 with a mean Event SNR of 2.91.

Effect of slot ordering. Role-aligned slots yield higher Event SNR than Role-shuffled slots on six of the eight held-out failure trajectories. A two-sided paired Wilcoxon signed-rank test does not establish significance ($p = 0.078$), so we read this as a trend rather than a conclusive gap.

Effect of simulation initialization. Despite its larger encoder-training budget, Real-only achieves a VOC of 53.1, Bump Ratio of 25.4, and Event SNR of -0.59, compared with 77.3, 13.5, and 2.91 for Sim+Real. Thus, the benefit cannot be attributed to additional optimization.

Effect of pseudo-label source. The Sim+Real SAM 2 and CoTracker models achieve comparable VOC (76.5 versus 77.3), but CoTracker reduces Bump Ratio from 17.9 to 13.5 and increases Event SNR from 0.91 to 2.91. SAM 2 yields a larger raw Event Response, whereas CoTracker has higher mean Event SNR. This comparison favors CoTracker for the evaluated combination of temporal consistency and object-drop sensitivity, without establishing a general advantage of sparse over dense pseudo-labels.

Temporal smoothing control. At $\alpha = 0.5$, causal EMA reduces DINO's Bump Ratio from 31.5 to 19.3, but its Event SNR increases only from 0.51 to 0.60. Role-aligned slots reach a lower Bump Ratio (13.5) and a higher Event SNR (2.91) without any smoothing, so post-hoc smoothing of a global representation does not recover both properties. EMA is an evaluation control; policy labels use values without output smoothing.

F. Real-Robot Policy Evaluation

Evaluation protocol. All policies are fine-tuned on the same 20 demonstrations using one side-view camera. The baseline uses unconditioned flow matching; the conditioned

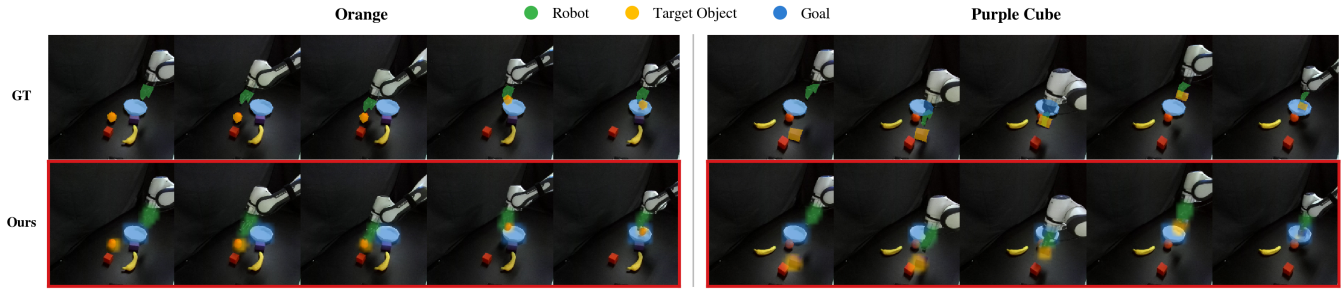


Fig. 4. Qualitative role grounding examples for Orange and Purple Cube. Predicted robot, target-object, and goal slots are shown in green, yellow, and blue.

TABLE IV

VALUE RELIABILITY AND OBJECT-DROP SENSITIVITY ON 16 HELD-OUT SUCCESSFUL AND EIGHT HELD-OUT FAILURE TRAJECTORIES (MEAN ± STANDARD DEVIATION ACROSS THREE VALUE-MODEL SEEDS). VOC IS SPEARMAN'S $\rho \times 100$; BUMP RATIO IS IN PERCENT, BUMP MAGNITUDE AND EVENT RESPONSE ARE SCALED BY 100, AND EVENT SNR IS UNITLESS. BOLDFACE MARKS THE BEST VALUE PER COLUMN.

Representation	Encoder Training	Label	Input Dim.	Successful Trajectories			Object-Drop Trajectories	
				VOC ↑	Bump Ratio ↓	Bump Mag. ↓	Event Response ↑	Event SNR ↑
DINO [6]	–	–	384	71.6 ± 3.1	31.5 ± 2.3	2.7 ± 1.0	6.3 ± 2.3	0.51 ± 0.10
R3M [7]	–	–	2048	60.0 ± 5.2	19.8 ± 1.8	0.9 ± 0.1	0.9 ± 0.7	0.03 ± 0.20
VIP [8]	–	–	1024	67.6 ± 2.1	14.3 ± 1.1	0.9 ± 0.1	−1.6 ± 0.7	0.04 ± 0.08
DINO-PCA	–	–	192	76.4 ± 2.7	31.9 ± 1.5	2.0 ± 0.4	7.6 ± 2.2	0.61 ± 0.10
R3M-PCA	–	–	192	64.2 ± 0.4	21.2 ± 0.7	1.1 ± 0.1	−0.2 ± 0.0	0.42 ± 0.02
VIP-PCA	–	–	192	76.0 ± 3.7	14.4 ± 1.5	**0.7 ± 0.2**	1.5 ± 0.6	0.43 ± 0.13
Role-aligned	Real only	SAM 2	192	51.4 ± 8.8	30.7 ± 1.6	4.1 ± 0.2	−4.3 ± 2.7	−0.27 ± 0.01
Role-aligned	Sim + Real	SAM 2	192	76.5 ± 1.3	17.9 ± 0.5	1.6 ± 0.1	16.7 ± 0.9	0.91 ± 0.05
Role-aligned	Real only	CoTracker	192	53.1 ± 6.2	25.4 ± 1.4	2.9 ± 0.4	−9.6 ± 0.3	−0.59 ± 0.10
Role-shuffled	Sim + Real	CoTracker	192	69.6 ± 3.0	26.7 ± 0.6	2.4 ± 0.3	**16.8 ± 0.2**	1.96 ± 0.75
Role-aligned (Ours)	Sim + Real	CoTracker	192	**77.3 ± 4.3**	**13.5 ± 1.4**	1.2 ± 0.3	13.3 ± 2.4	**2.91 ± 1.48**
Temporal smoothing control								
DINO + Causal EMA ($\alpha = 0.5$)	–	–	384	71.3 ± 4.7	19.3 ± 2.6	1.1 ± 0.4	6.1 ± 2.3	0.60 ± 0.13

TABLE V

REAL-ROBOT SUCCESS RATE (%, MEAN ± STANDARD DEVIATION ACROSS THREE EVALUATION SEEDS). BOLDFACE MARKS THE BEST RESULT PER TASK.

Policy	Value Rep.	Task				Avg.
		Banana	Orange	Red	Purple	
$\pi_{0.5}$	–	25.0 ± 0.0	13.3 ± 2.9	26.7 ± 10.4	28.3 ± 10.4	23.3 ± 5.1
	Image	38.3 ± 2.9	25.0 ± 5.0	35.0 ± 13.2	31.7 ± 7.6	32.5 ± 2.2
	Role Slots	**60.0 ± 5.0**	**30.0 ± 5.0**	**43.3 ± 7.6**	**46.7 ± 7.6**	**45.0 ± 3.3**

variants use the same combined conditioned and unconditioned objective with labels derived from pooled DINO features or role slots. For each method, we evaluate one trained policy over 20 trials per task for each of three inference seeds, totaling 240 trials. Policies are not retrained between seeds.

EMA model tracking is disabled, and each policy is trained for 30,000 steps with batch size 16. This data-limited, single-view setting constrains absolute performance.

Advantage-label analysis. Positive-condition rates are 44.2% for role slots and 50.3% for image values, with mean absolute advantages of 0.0680 and 0.0486 before instruction-wise standardization on the $[-1,0]$ return scale. Both label sources exhibit elevated final-quintile positive rates, 63.2% for role slots and 68.9% for image. A leave-one-trajectory-out decile predictor recovers the conditions with Cohen's $\kappa = 0.34$ for role slots and 0.26 for image. Positive rates stay between 28.7% and 60.5% in eight of ten deciles for role slots and between 20.4% and 64.8% in nine image deciles. Thus, both signals are only partially explained by trajectory position, and the two orderings disagree: the image labels are more late-concentrated yet less predictable from position, and they yield lower policy success. Neither timing measure tracks the performance gap. We do not train a position-only policy, so the effect of label timing is not isolated.

Results. Advantages derived from role slots produce the highest success on all four variants. Pooling these trials, they achieve 45.0% success (Wilson 95% CI: $[38.8, 51.4]$%), compared with 32.5% ($[26.9, 38.7]$%) for Image Value and 23.3% ($[18.4, 29.1]$%) without conditioning. The outcomes for role slots and image values differ under a two-sided Fisher exact test ($p < 0.01$). The reported intervals and test characterize rollout variability for the evaluated policies. Variability across independent training runs is not measured. The ordering is consistent across all four task variants for the evaluated policies.

VI. CONCLUSION

We studied simulation-pretrained role representations for real-world value learning across four target-object variants of a pick-and-place task. The evaluated role slots combine temporal value consistency with object-drop sensitivity, and their offline conditions improve policy performance. Comparisons with SAM 2-derived supervision and Real-only encoders characterize the effects of pseudo-label source and simulation

initialization. Remaining limitations include that we do not isolate the contribution of recurrent memory, that failure sensitivity is evaluated only on object drops, and that the policy results do not capture variation across training runs.

REFERENCES

[1] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2017, pp. 23–30.

[2] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *IEEE International Conference on Robotics and Automation*, 2018, pp. 3803–3810.

[3] S. James, P. Wohlhart, M. Kalakrishnan, D. Kalashnikov, A. Irpan, J. Ibarz, S. Levine, R. Hadsell, and K. Bousmalis, "Sim-to-real via sim-to-sim: Data-efficient robotic grasping via randomized-to-canonical adaptation networks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 12 627–12 637.

[4] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess *et al.*, "$\pi_{0.5}$: A vision-language-action model with open-world generalization," in *Conference on Robot Learning*, 2025.

[5] Physical Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley *et al.*, "$\pi^*_{0.6}$: a VLA that learns from experience," *arXiv preprint arXiv:2511.14759*, 2025.

[6] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, "Emerging properties in self-supervised vision transformers," in *IEEE/CVF International Conference on Computer Vision*, 2021, pp. 9650–9660.

[7] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta, "R3M: A universal visual representation for robot manipulation," in *Conference on Robot Learning*, 2022.

[8] Y. J. Ma, S. Sodhani, D. Jayaraman, O. Bastani, V. Kumar, and A. Zhang, "VIP: Towards universal visual reward and representation via value-implicit pre-training," in *International Conference on Learning Representations*, 2023.

[9] F. Locatello, D. Weissenborn, T. Unterthiner, A. Mahendran, G. Heigold, J. Uszkoreit, A. Dosovitskiy, and T. Kipf, "Object-centric learning with slot attention," in *Advances in Neural Information Processing Systems*, 2020.

[10] T. Kipf, G. F. Elsayed, A. Mahendran, A. Stone, S. Sabour, G. Heigold, R. Jonschkowski, A. Dosovitskiy, and K. Greff, "Conditional object-centric learning from video," in *International Conference on Learning Representations*, 2022.

[11] M. Seitzer, M. Horn, A. Zadaianchuk, D. Zietlow, T. Xiao, C.-J. Simon-Gabriel, T. He, Z. Zhang, B. Schölkopf, T. Brox, and F. Locatello, "Bridging the gap to real-world object-centric learning," in *International Conference on Learning Representations*, 2023.

[12] N. Karaev, I. Rocco, B. Graham, N. Neverova, A. Vedaldi, and C. Rupprecht, "CoTracker: It is better to track together," in *European Conference on Computer Vision*. Springer, 2024, pp. 18–35.

[13] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang *et al.*, "PaliGemma: A versatile 3B VLM for transfer," *arXiv preprint arXiv:2407.07726*, 2024.

[14] K. Bousmalis, A. Irpan, P. Wohlhart, Y. Bai, M. Kelcey, M. Kalakrishnan, L. Downs, J. Ibarz, P. Pastor, K. Konolige, S. Levine, and V. Vanhoucke, "Using simulation and domain adaptation to improve efficiency of deep robotic grasping," in *IEEE International Conference on Robotics and Automation*, 2018, pp. 4243–4250.

[15] S. Ross, G. J. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *International Conference on Artificial Intelligence and Statistics*, 2011, pp. 627–635.

[16] A. Zadaianchuk, M. Seitzer, and G. Martius, "Object-centric learning for real-world videos by predicting temporal feature similarities," in *Advances in Neural Information Processing Systems*, 2023.

[17] A. Manasyan, M. Seitzer, F. Radovic, G. Martius, and A. Zadaianchuk, "Temporally consistent object-centric learning by contrasting slots," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.

[18] G. F. Elsayed, A. Mahendran, S. van Steenkiste, K. Greff, M. C. Mozer, and T. Kipf, "SAVi++: Towards end-to-end object-centric learning from real-world videos," in *Advances in Neural Information Processing Systems*, 2022.

[19] R. Veerapaneni, J. D. Co-Reyes, M. Chang, M. Janner, C. Finn, J. Wu, J. B. Tenenbaum, and S. Levine, "Entity abstraction in visual model-based reinforcement learning," in *Conference on Robot Learning*, 2019.

[20] Z. Wu, N. Dvornik, K. Greff, T. Kipf, and A. Garg, "SlotFormer: Unsupervised visual dynamics simulation with object-centric models," in *International Conference on Learning Representations*, 2023.

[21] A. Zadaianchuk, M. Seitzer, and G. Martius, "Self-supervised visual reinforcement learning with object-centric representations," in *International Conference on Learning Representations*, 2021.

[22] M. Shridhar, L. Manuelli, and D. Fox, "CLIPort: What and where pathways for robotic manipulation," in *Conference on Robot Learning*, 2021.

[23] A. Stone, T. Xiao, Y. Lu, K. Gopalakrishnan, K.-H. Lee, Q. Vuong, P. Wohlhart, S. Kirmani, B. Zitkovich, F. Xia, C. Finn, and K. Hausman, "Open-world object manipulation using pre-trained vision-language models," in *Conference on Robot Learning*, 2023.

[24] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *European Conference on Computer Vision*, 2024.

[25] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna *et al.*, "OpenVLA: An open-source vision-language-action model," in *Conference on Robot Learning*, 2024.

[26] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn *et al.*, "π_0: A vision-language-action flow model for general robot control," in *Robotics: Science and Systems*, 2025.

[27] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline reinforcement learning: Tutorial, review, and perspectives on open problems," *arXiv preprint arXiv:2005.01643*, 2020.

[28] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, "Decision transformer: Reinforcement learning via sequence modeling," in *Advances in Neural Information Processing Systems*, 2021.

[29] I. Kostrikov, A. Nair, and S. Levine, "Offline reinforcement learning with implicit Q-learning," in *International Conference on Learning Representations*, 2022.

[30] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, and C. Feichtenhofer, "SAM 2: Segment anything in images and videos," *arXiv preprint arXiv:2408.00714*, 2024. [Online]. Available: https://arxiv.org/abs/2408.00714

[31] H. Blum, P.-E. Sarlin, J. Nieto, R. Siegwart, and C. Cadena, "Fishyscapes: A benchmark for safe semantic segmentation in autonomous driving," in *IEEE/CVF International Conference on Computer Vision Workshops*, 2019, pp. 2403–2412.

[32] R. Chan, K. Lis, S. Uhlemeyer, H. Blum, S. Honari, R. Siegwart, P. Fua, M. Salzmann, and M. Rottmann, "SegmentMeIfYouCan: A benchmark for anomaly segmentation," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, vol. 1, 2021.

[33] Y. J. Ma, J. Hejna, C. Fu, D. Shah, J. Liang, Z. Xu, S. Kirmani, P. Xu, D. Driess, T. Xiao, O. Bastani, D. Jayaraman, W. Yu, T. Zhang, D. Sadigh, and F. Xia, "Vision language models are in-context value learners," in *International Conference on Learning Representations*, 2025.